Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

AutoAudit

Sistema de auditoría automatizado

Documentación técnica — Proyecto del curso

Nombre del estudiante
Carné
02/11/2026

1. Descripción general y arquitectura

AutoAudit es un sistema de auditoría configurable para PostgreSQL. El administrador selecciona, a partir del diccionario de datos del DBMS, qué tablas y columnas deben monitorearse y qué eventos (INSERT, UPDATE, DELETE) registrar en cada columna. A partir de esa configuración el sistema **genera automáticamente** una función PL/pgSQL y un trigger `AFTER ... FOR EACH ROW` exclusivos para cada tabla, que registran cada modificación en una bitácora centralizada, normalizada e inmutable.

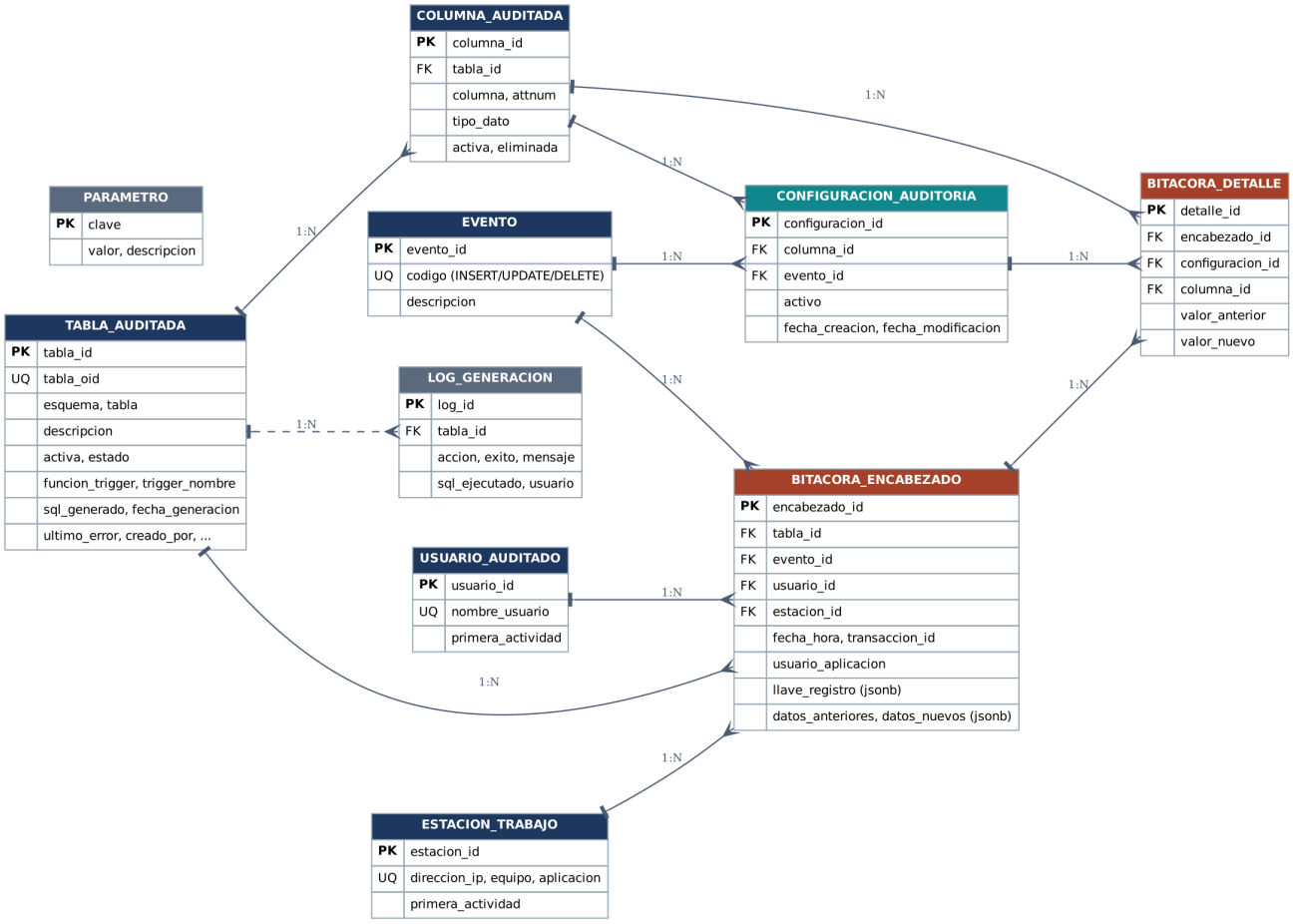
Componente	Tecnología	Ubicación
Base de datos, generador de triggers, bitácora, consultas	PostgreSQL 16.15 (Ubuntu 16.15-0ubuntu0.24.04.1) (proyecto: PostgreSQL 18) · PL/pgSQL	`sql/`
Servidor de aplicación (API REST)	Python 3 · FastAPI · psycopg 3	`app/main.py`
Interfaz web	HTML5, CSS, JavaScript (sin dependencias externas)	`app/static/`
Reportes	CSV, Excel (openpyxl), PDF (reportlab)	`/api/bitacora/exportar`
Pruebas de validación	Scripts SQL con aserciones	`pruebas/`
Sistema operativo	Ubuntu 26.04 LTS	`install.sh`, `run.sh`

Flujo de funcionamiento

1. El administrador consulta el diccionario (`v_dic_tablas`, `v_dic_columnas`) y elige tablas, columnas y eventos.
2. La configuración se guarda en `tabla_auditada`, `columna_auditada` y `configuracion_auditoria`, validando existencia contra `pg_catalog`.
3. `fn_generar_trigger` construye el código SQL con las columnas y eventos “compilados” y lo ejecuta.
4. Cada INSERT/UPDATE/DELETE de los usuarios dispara el trigger, que escribe un encabezado (operación) y un detalle por columna afectada.
5. Los auditores consultan el historial por usuario, por columna o por registro y exportan reportes.

Si la estructura de una tabla auditada cambia (RENAME, DROP COLUMN … CASCADE, DROP TABLE), dos *event triggers* sincronizan la configuración y regeneran el trigger automáticamente.

2. Diagrama entidad-relación



Relaciones

Relación	Cardinalidad	Significado
tabla_auditada → columna_auditada	1:N	Una tabla auditada tiene varias columnas seleccionadas.
columna_auditada → configuracion_auditoria ← evento	N:M resuelta	La configuración indica qué eventos se registran ante el cambio de cada columna de cada tabla.
tabla_auditada, evento, usuario_auditado, estacion_trabajo → bitacora_encabezado	1:N	Cada operación registrada referencia la tabla, el evento, el usuario responsable y la estación de trabajo.
bitacora_encabezado → bitacora_detalle	1:N	Una operación afecta una o varias columnas.
configuracion_auditoria → bitacora_detalle	1:N	Cada registro de bitácora referencia la configuración específica usada.
tabla_auditada → log_generacion	1:N	Historial de generación de triggers y errores.

> Las llaves foráneas de la bitácora usan `ON DELETE RESTRICT`: una configuración con historial no puede borrarse, solo desactivarse. Si la tabla origen se elimina, la configuración queda en estado **HUERFANA** y la bitácora (con la fila completa en `datos_anteriores`) se conserva.

3. Diccionario de datos

Generado automáticamente desde `pg_catalog` (ver `docs/diccionario_datos.sql`). Esquema: `auditoria`.

evento

Catálogo de operaciones DML que pueden auditarse

Columna	Tipo	Nulo	Llave	Defecto	Descripción
evento_id	smallint	NO	PK		Identificador del evento (1 INSERT, 2 UPDATE, 3 DELETE)
codigo	character varying(10)	NO	UQ		Código de la operación DML
descripcion	character varying(100)	NO			Descripción legible del evento

Restricciones CHECK: `CHECK (((codigo)::text = ANY ((ARRAY['INSERT'::character varying, 'UPDATE'::character varying, 'DELETE'::character varying])::text[])))`

Índices: `evento_codigo_key`

tabla_auditada

Tablas origen configuradas para auditoría (una fila = un trigger)

Columna	Tipo	Nulo	Llave	Defecto	Descripción
tabla_id	integer	NO	PK	`nextval('auditoria.tabla_auditada_tabla_id_seq'::regclass)`	Identificador de la configuración de la tabla
tabla_oid	oid	NO	UQ		OID de la tabla en pg_class; identifica la tabla aunque se renombre
esquema	name	NO			Esquema de la tabla origen
tabla	name	NO			Nombre de la tabla origen
descripcion	character varying(250)	SÍ			Motivo o descripción de la auditoría
activa	boolean	NO		`true`	Interruptor maestro: si es falso no existe trigger
estado	character varying(15)	NO		`'PENDIENTE'::character varying`	PENDIENTE, ACTIVO, INACTIVO, ERROR o HUERFANA (tabla origen eliminada)
funcion_trigger	name	SÍ			Nombre de la función PL/pgSQL generada en el esquema auditoria
trigger_nombre	name	SÍ			Nombre del trigger generado en la tabla origen (trg_autoaudit)
sql_generado	text	SÍ			Último código SQL generado y aplicado
fecha_generacion	timestamp with time zone	SÍ			Fecha y hora de la última generación exitosa
ultimo_error	text	SÍ			Último error de validación o generación
creado_por	name	NO		`SESSION_USER`	Usuario que registró la tabla
fecha_creacion	timestamp with time zone	NO		`now()`	Fecha de registro
modificado_por	name	SÍ			Último usuario que modificó la configuración
fecha_modificacion	timestamp with time zone	SÍ			Fecha de la última modificación

Restricciones CHECK: `CHECK (((estado)::text = ANY ((ARRAY['PENDIENTE'::character varying, 'ACTIVO'::character varying, 'INACTIVO'::character varying, 'ERROR'::character varying, 'HUERFANA'::character varying])::text[])))`

Índices: `tabla_auditada_tabla_oid_key`; `uq_tabla_auditada_nombre`

columna_auditada

Columnas de cada tabla auditada

Columna	Tipo	Nulo	Llave	Defecto	Descripción
columna_id	integer	NO	PK	`nextval('auditoria.columna_auditada_columna_id_seq'::regclass)`	Identificador de la columna auditada
tabla_id	integer	NO	FK→tabla_auditada UQ		Tabla a la que pertenece (FK tabla_auditada)
columna	name	NO			Nombre actual de la columna
attnum	smallint	NO	UQ		Número de columna en pg_attribute; sobrevive a un RENAME

tipo_dato	text	NO			Tipo de dato según el diccionario
activa	boolean	NO		`true`	Indica si la columna participa en la auditoría
eliminada	boolean	NO		`false`	La columna ya no existe en la tabla origen (DROP COLUMN)
fecha_creacion	timestamp with time zone	NO		`now()`	Fecha de registro

Índices: `uq_columna_auditada`

configuracion_auditoria

Eventos a auditar por cada columna de cada tabla

Columna	Tipo	Nulo	Llave	Defecto	Descripción
configuracion_id	integer	NO	PK	`nextval('auditoria.configuracion_auditoria_configuracion_id_seq'::regclass)`	Identificador de la configuración columna-evento
columna_id	integer	NO	FK→columna_auditada UQ		Columna configurada (FK columna_auditada)
evento_id	smallint	NO	FK→evento UQ		Evento que se audita (FK evento)
activo	boolean	NO		`true`	Si el evento se registra para esa columna
fecha_creacion	timestamp with time zone	NO		`now()`	Fecha de creación
fecha_modificacion	timestamp with time zone	SÍ			Fecha de la última activación/desactivación

Índices: `uq_configuracion`

usuario_auditado

Usuarios de base de datos detectados en la bitácora

Columna	Tipo	Nulo	Llave	Defecto	Descripción
usuario_id	integer	NO	PK	`nextval('auditoria.usuario_auditado_usuario_id_seq'::regclass)`	Identificador del usuario auditado
nombre_usuario	name	NO	UQ		Usuario de PostgreSQL (session_user) que realizó cambios
primera_actividad	timestamp with time zone	NO		`now()`	Primera vez que aparece en la bitácora

Índices: `usuario_auditado_nombre_usuario_key`

estacion_trabajo

Estaciones de trabajo desde donde se originan los cambios

Columna	Tipo	Nulo	Llave	Defecto	Descripción
estacion_id	integer	NO	PK	`nextval('auditoria.estacion_trabajo_estacion_id_seq'::regclass)`	Identificador de la estación
direccion_ip	character varying(64)	NO	UQ		IP de la conexión (inet_client_addr), "local" si es socket, según política de privacidad
equipo	character varying(128)	NO	UQ	`'desconocido'::character varying`	Nombre del equipo informado por la aplicación (autoaudit.equipo)
aplicacion	character varying(128)	NO	UQ	`'desconocida'::character varying`	application_name de la conexión
primera_actividad	timestamp with time zone	NO		`now()`	Primera vez que aparece en la bitácora

Índices: `uq_estacion`

bitacora_encabezado

Bitácora: una fila por registro afectado. Conserva la fila completa aunque el origen sea eliminado

Columna	Tipo	Nulo	Llave	Defecto	Descripción

encabezado_id	bigint	NO	PK	`nextval('auditoria.bitacora_encabezado_encabezado_id_seq'::regclass)`	Identificador de la operación
fecha_hora	timestamp with time zone	NO		`clock_timestamp()`	Fecha y hora exacta del cambio
tabla_id	integer	NO	FK→tabla_auditada		Tabla afectada (FK tabla_auditada)
evento_id	smallint	NO	FK→evento		Operación realizada (FK evento)
usuario_id	integer	NO	FK→usuario_auditado		Usuario responsable (FK usuario_auditado)
estacion_id	integer	NO	FK→estacion_trabajo		Estación de origen (FK estacion_trabajo)
usuario_aplicacion	character varying(128)	SÍ			Usuario de la aplicación, si la aplicación lo informa
transaccion_id	bigint	NO		`txid_current()`	Identificador de la transacción (txid)
llave_registro	jsonb	SÍ			Llave primaria del registro afectado en formato JSON
datos_anteriores	jsonb	SÍ			Fila completa antes del cambio (UPDATE/DELETE)
datos_nuevos	jsonb	SÍ			Fila completa después del cambio (INSERT/UPDATE)

Índices: `ix_enc_fecha`; `ix_enc_usuario_fecha`; `ix_enc_tabla_fecha`; `ix_enc_estacion`; `ix_enc_llave`

bitacora_detalle

Bitácora: valor anterior y nuevo por columna afectada

Columna	Tipo	Nulo	Llave	Defecto	Descripción
detalle_id	bigint	NO	PK	`nextval('auditoria.bitacora_detalle_detalle_id_seq'::regclass)`	Identificador del detalle
encabezado_id	bigint	NO	FK→bitacora_encabezado		Operación a la que pertenece (FK bitacora_encabezado)
configuracion_id	integer	NO	FK→configuracion_auditoria		Configuración columna-evento usada (FK configuracion_auditoria)
columna_id	integer	NO	FK→columna_auditada		Columna afectada (FK columna_auditada)
valor_anterior	text	SÍ			Valor antes del cambio, como texto
valor_nuevo	text	SÍ			Valor después del cambio, como texto

Índices: `ix_det_encabezado`; `ix_det_columna`; `ix_det_configuracion`

log_generacion

Registro de acciones del generador de triggers y errores

Columna	Tipo	Nulo	Llave	Defecto	Descripción
log_id	bigint	NO	PK	`nextval('auditoria.log_generacion_log_id_seq'::regclass)`	Identificador del registro
fecha_hora	timestamp with time zone	NO		`clock_timestamp()`	Fecha y hora de la acción
tabla_id	integer	SÍ	FK→tabla_auditada		Tabla relacionada (FK tabla_auditada)
objeto	text	SÍ			Objeto afectado (función o columna)
accion	character varying(20)	NO			GENERAR, ELIMINAR, VALIDAR, SINCRONIZAR, DDL o PURGAR
exito	boolean	NO			Resultado de la acción
mensaje	text	SÍ			Mensaje o error
sql_ejecutado	text	SÍ			Código SQL ejecutado
usuario	name	NO		`SESSION_USER`	Usuario que ejecutó la acción

parametro

Parámetros globales: privacidad de IP y retención

Columna	Tipo	Nulo	Llave	Defecto	Descripción
clave	character varying(50)	NO	PK		Nombre del parámetro

valor	character varying(200)	NO			Valor configurado
descripcion	character varying(300)	SÍ			Descripción del parámetro

Vistas

Vista	Descripción
v_bitacora	Bitácora desnormalizada para consulta y reportes
v_dic_columnas	Diccionario: columnas de las tablas de usuario
v_dic_llaves	Diccionario: llaves primarias, foráneas y únicas
v_dic_tablas	Diccionario: tablas de usuario existentes
v_dic_triggers	Diccionario: triggers definidos por usuarios
v_estado_auditoria	Estado actual de la auditoría por tabla
v_registros_eliminados	
v_resumen_usuarios	

Funciones

Función	Retorna	Seguridad	Descripción
fn_activar_tabla(p_tabla_id integer, p_activa boolean)	jsonb	DEFINER	Activa (genera trigger) o desactiva (elimina trigger) una tabla.
fn_configurar_columna(p_esquema text, p_tabla text, p_columna text, p_insert boolean, p_update boolean, p_delete boolean)	integer	DEFINER	Valida la columna y guarda qué eventos (INSERT/UPDATE/DELETE) se auditan.
fn_configurar_tabla(p_esquema text, p_tabla text, p_columnas text[], p_insert boolean, p_update boolean, p_delete boolean, p_aplicar boolean)	jsonb	DEFINER	Atajo: configura varias columnas y genera el trigger.
fn_construir_sql_trigger(p_tabla_id integer)	text	INVOKER	Construye el código del trigger sin ejecutarlo (vista previa).
fn_consultar_bitacora(p_usuario text, p_esquema text, p_tabla text, p_columna text, p_operacion text, p_desde timestamp with time zone, p_hasta timestamp with time zone, p_limite integer)	SETOF auditoria.v_bitacora	INVOKER	Consulta parametrizada con filtros opcionales.
fn_crear_encabezado(p_tabla_id integer, p_evento_id smallint, p_anterior jsonb, p_nuevo jsonb, p_llave jsonb)	bigint	INVOKER	Inserta el encabezado de bitácora con usuario y estación. Solo autoaudit_owner.
fn_dic_llave_primaria(p_oid oid)	name[]	INVOKER	Columnas de la llave primaria de una tabla.
fn_dic_oid_tabla(p_esquema text, p_tabla text)	oid	INVOKER	OID de una tabla a partir de esquema y nombre.
fn_eliminar_configuracion(p_tabla_id integer)	jsonb	DEFINER	Elimina la configuración si no tiene bitácora.
fn_eliminar_trigger(p_tabla_id integer)	jsonb	DEFINER	Elimina el trigger y la función generada.
fn_eliminar_trigger_interno(p_tabla_id integer)	jsonb	DEFINER	Implementación interna; no se otorga a usuarios.
fn_es_admin()	boolean	INVOKER	Indica si el usuario de sesión es administrador.
fn_estadisticas(p_dias integer)	jsonb	INVOKER	Indicadores para el panel principal.
fn_evt_ddl_end()	event_trigger	DEFINER	Event trigger: tras ALTER TABLE sincroniza y regenera.
fn_evt_sql_drop()	event_trigger	DEFINER	Event trigger: detecta DROP TABLE / DROP COLUMN.
fn_generar_trigger(p_tabla_id integer)	jsonb	DEFINER	Valida, construye y ejecuta el trigger; registra en log_generacion.
fn_generar_trigger_interno(p_tabla_id integer)	jsonb	DEFINER	Implementación interna (sin verificación de rol); no se otorga a usuarios.
fn_historial_columna(p_esquema text, p_tabla text, p_columna text, p_desde timestamp with time zone, p_hasta timestamp with time zone)	SETOF auditoria.v_bitacora	INVOKER	Todos los cambios de una columna.
fn_historial_registro(p_esquema text, p_tabla text, p_llave jsonb)	SETOF auditoria.v_bitacora	INVOKER	Línea de tiempo de un registro por su llave.
fn_historial_usuario(p_usuario text, p_desde timestamp with time zone, p_hasta timestamp with time zone)	SETOF auditoria.v_bitacora	INVOKER	Historial completo de un usuario.
fn_ip_cliente()	text	INVOKER	IP de origen aplicando la política de privacidad.
fn_proteger_bitacora()	trigger	INVOKER	Trigger que impide UPDATE/DELETE/TRUNCATE en la bitácora.
fn_purgar_bitacora()	bigint	INVOKER	Depura según dias_retencion (solo superusuario).

fn_regenerar_todos()	TABLE(tabla_id integer, tabla text, exito boolean, mensaje text)	DEFINER	Regenera los triggers de todas las tablas activas.
fn_registrar_tabla(p_esquema text, p_tabla text, p_descripcion text)	integer	DEFINER	Valida en el diccionario y registra una tabla para auditoría.
fn_set_parametro(p_clave text, p_valor text)	void	DEFINER	Modifica un parámetro global.
fn_sincronizar_tabla(p_tabla_id integer)	void	DEFINER	Actualiza nombres por OID/attnum y marca columnas eliminadas.
fn_validar_configuracion(p_tabla_id integer)	TABLE(nivel text, mensaje text)	INVOKER	Compara la configuración contra el diccionario (ERROR/AVISO).
fn_verificar_admin()	void	INVOKER	Lanza error 42501 si el usuario no es administrador.

4. Manual de configuración y activación de auditoría por tabla

Solo los miembros del rol `autoaudit_admin` (usuario de ejemplo `admin_auditoria`) pueden configurar.

4.1 Desde la interfaz web

1. Iniciar la aplicación (`bash run.sh`) y abrir `http://localhost:8000`. Ingresar como `admin_auditoria`.
2. Menú **Configuración**: a la izquierda aparece el diccionario de datos con el estado de cada tabla.
3. Seleccionar la tabla. Se muestran sus columnas (tipo, PK/FK, NOT NULL) con tres casillas: INSERT, UPDATE y DELETE. Al hacer clic en el encabezado de un evento se marcan/desmarcan todas las columnas.
4. Escribir opcionalmente una descripción y pulsar **Guardar y generar trigger**. El sistema valida, genera el código, lo ejecuta y muestra la vista previa del SQL. **Guardar sin aplicar** solo guarda la configuración (estado PENDIENTE).
5. **Ver código SQL** muestra la validación contra el diccionario y el código que se generará.
6. **Desactivar** elimina el trigger conservando la configuración y la bitácora; **Activar** lo vuelve a generar. **Regenerar** reconstruye el trigger; **Eliminar config.** solo aparece si la tabla no tiene bitácora.

4.2 Desde SQL

```
-- Consultar el diccionario
SELECT * FROM auditoria.v_dic_tablas;
SELECT * FROM auditoria.v_dic_columnas WHERE esquema='tienda' AND tabla='productos';

-- Configuración granular (INSERT, UPDATE, DELETE) y generación
SELECT auditoria.fn_configurar_columna('tienda','productos','precio',     true,  true, true);
SELECT auditoria.fn_configurar_columna('tienda','productos','existencia', false, true, false);
SELECT auditoria.fn_generar_trigger(tabla_id) FROM auditoria.tabla_auditada
 WHERE esquema='tienda' AND tabla='productos' AND estado<>'HUERFANA';

-- Atajo: varias columnas con los mismos eventos y aplicar
SELECT auditoria.fn_configurar_tabla('tienda','clientes', ARRAY['nombre','email'], true, true, true);

-- Activar / desactivar / regenerar / estado
SELECT auditoria.fn_activar_tabla(1, false);
SELECT auditoria.fn_activar_tabla(1, true);
SELECT * FROM auditoria.fn_regenerar_todos();
SELECT * FROM auditoria.v_estado_auditoria;
```

4.3 Estados de una tabla

Estado	Significado
PENDIENTE	Configuración guardada o modificada, falta generar el trigger.
ACTIVO	Trigger generado y vigente.
INACTIVO	Desactivada: sin trigger, se conserva configuración y bitácora.
ERROR	La validación o la generación falló; el detalle está en `ultimo_error` y en `log_generacion`.
HUERFANA	La tabla origen fue eliminada; se conserva su historial.

4.4 Ejemplo de código generado

```
-- ================================================================
-- Código generado automáticamente por AutoAudit. NO EDITAR A MANO.
-- Tabla origen : tienda.productos   (tabla_id = 2)
-- Generado     : 2026-11-02 16:44:56 por postgres
-- ================================================================
CREATE OR REPLACE FUNCTION auditoria.fn_aud_2_tienda_productos()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = pg_catalog, auditoria
AS $autoaudit$
DECLARE
    v_enc bigint;
BEGIN
    IF TG_OP = 'INSERT' THEN
        v_enc := auditoria.fn_crear_encabezado(2, 1::smallint, NULL, to_jsonb(NEW), jsonb_build_object('producto_id', NEW.producto_id));
        INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id, valor_anterior, valor_nuevo)
        VALUES (v_enc, 17, 6, NULL, NEW.nombre::text),
               (v_enc, 20, 7, NULL, NEW.precio::text);
        RETURN NULL;
    END IF;
    IF TG_OP = 'UPDATE' THEN
        IF OLD.nombre IS DISTINCT FROM NEW.nombre THEN
            v_enc := COALESCE(v_enc, auditoria.fn_crear_encabezado(2, 2::smallint, to_jsonb(OLD), to_jsonb(NEW),
jsonb_build_object('producto_id', NEW.producto_id)));
            INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id, valor_anterior, valor_nuevo)
            VALUES (v_enc, 16, 6, OLD.nombre::text, NEW.nombre::text);
        END IF;
        IF OLD.precio IS DISTINCT FROM NEW.precio THEN
            v_enc := COALESCE(v_enc, auditoria.fn_crear_encabezado(2, 2::smallint, to_jsonb(OLD), to_jsonb(NEW),
jsonb_build_object('producto_id', NEW.producto_id)));
            INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id, valor_anterior, valor_nuevo)
            VALUES (v_enc, 19, 7, OLD.precio::text, NEW.precio::text);
        END IF;
        IF OLD.existencia IS DISTINCT FROM NEW.existencia THEN
            v_enc := COALESCE(v_enc, auditoria.fn_crear_encabezado(2, 2::smallint, to_jsonb(OLD), to_jsonb(NEW),
jsonb_build_object('producto_id', NEW.producto_id)));
            INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id, valor_anterior, valor_nuevo)
            VALUES (v_enc, 22, 8, OLD.existencia::text, NEW.existencia::text);
        END IF;
        RETURN NULL;
    END IF;
    IF TG_OP = 'DELETE' THEN
        v_enc := auditoria.fn_crear_encabezado(2, 3::smallint, to_jsonb(OLD), NULL, jsonb_build_object('producto_id', OLD.producto_id));
        INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id, valor_anterior, valor_nuevo)
        VALUES (v_enc, 18, 6, OLD.nombre::text, NULL),
               (v_enc, 21, 7, OLD.precio::text, NULL);
        RETURN NULL;
    END IF;
    RETURN NULL;
END;
$autoaudit$;

ALTER FUNCTION auditoria.fn_aud_2_tienda_productos() OWNER TO autoaudit_owner;
REVOKE ALL ON FUNCTION auditoria.fn_aud_2_tienda_productos() FROM PUBLIC;
COMMENT ON FUNCTION auditoria.fn_aud_2_tienda_productos() IS 'AutoAudit: trigger de auditoría de tienda.productos';

CREATE OR REPLACE TRIGGER trg_autoaudit
    AFTER INSERT OR UPDATE OF nombre, precio, existencia OR DELETE
    ON tienda.productos
    FOR EACH ROW
    EXECUTE FUNCTION auditoria.fn_aud_2_tienda_productos();
```

Características del código generado: una función por tabla (requisito: cada trigger maneja exclusivamente una tabla origen), columnas y eventos fijos en el código (sin consultar la configuración por fila), `UPDATE OF columnas` para no dispararse con columnas no auditadas, `IS DISTINCT FROM` para registrar solo cambios reales (incluyendo NULL), `SECURITY DEFINER` con dueño `autoaudit_owner` y `search_path` fijo.

5. Procedimiento de instalación y despliegue de triggers

5.1 Requisitos

- Ubuntu 26.04 LTS con acceso `sudo`.
- PostgreSQL 18 (lo instala el script si no existe).
- Python 3 con `venv` (incluido en Ubuntu).

5.2 Instalación automática

```
cd autoaudit
sudo bash install.sh          # instala todo + datos de demostración
bash run.sh                   # inicia la aplicación en http://localhost:8000
sudo bash pruebas/ejecutar_pruebas.sh   # ejecuta las pruebas de validación
```

5.3 Orden de los scripts de base de datos

Script	Contenido
00_roles_y_bd.sql	Roles autoaudit_owner/admin/consulta, usuarios de demostración, base autoaudit_db.
01_esquema.sql	Tablas del sistema, índices, catálogo de eventos y parámetros.
02_diccionario.sql	Vistas de consulta al diccionario de datos del DBMS.
03_funciones_config.sql	Módulo de configuración con validaciones.
04_generador_triggers.sql	Generador dinámico de triggers y registro en bitácora.
05_consultas.sql	Consultas parametrizadas de historial y estadísticas.
06_seguridad.sql	Permisos y protección (inmutabilidad) de la bitácora.
07_eventos_ddl.sql	Event triggers para dependencias (DROP/ALTER).
08_datos_demo.sql	Esquema de ejemplo `tienda`.
09_comentarios.sql	Descripciones para el diccionario de datos.

5.4 Despliegue de triggers en una tabla propia

1. Crear la tabla en cualquier esquema (se recomienda que tenga llave primaria).
2. Configurarla desde la interfaz o con `fn_configurar_columna` / `fn_configurar_tabla`.
3. Generar con `fn_generar_trigger`. Verificar con `SELECT * FROM auditoria.v_estado_auditoria` (columna `sincronizado = true`) y en `auditoria.v_dic_triggers`.
4. Los errores quedan en `auditoria.log_generacion`; si la generación falla, el trigger anterior permanece intacto.

6. Seguridad

Requisito	Implementación
Solo administradores configuran	Funciones de configuración verifican `pg_has_role(session_user,'autoaudit_admin')` (error 42501). Los administradores no tienen INSERT/UPDATE/DELETE directo sobre las tablas de configuración.
Sin escritura directa en bitácora	Solo `autoaudit_owner` (rol NOLOGIN dueño de las funciones generadas) tiene INSERT. Un trigger de protección impide UPDATE, DELETE y TRUNCATE incluso al superusuario (salvo depuración explícita).
Privacidad de la IP	Parámetro `ip_modo`: completa, anonimizada (IPv4 /24, IPv6 /48) o ninguna. La IP se toma de `inet_client_addr()` (no la puede falsificar el cliente).
Usuario responsable	Se usa `session_user`, que no cambia con `SET ROLE` ni dentro de funciones SECURITY DEFINER.
Inyección SQL	El generador usa `format()` con `%I`/`%L`; la API usa parámetros y `psycopg.sql.Identifier`.
Retención	`dias_retencion` y `fn_purgar_bitacora()` (solo superusuario).

7. Pruebas de validación

Ejecutadas con `sudo bash pruebas/ejecutar_pruebas.sh`.

Resultado: **39 de 39 pruebas aprobadas**.

#	Caso de prueba	Resultado
1	INSERT genera encabezado en bitácora	**PASA**
2	INSERT registra la llave primaria del registro	**PASA**
3	INSERT guarda la fila completa nueva (datos_nuevos)	**PASA**
4	INSERT registra exactamente las 3 columnas configuradas	**PASA**
5	INSERT guarda valor nuevo de saldo = 1500.00	**PASA**
6	INSERT no registra columnas no configuradas (notas)	**PASA**
7	INSERT registra el usuario responsable (vendedor)	**PASA**
8	INSERT registra estación de trabajo (equipo y aplicación)	**PASA**
9	INSERT registra fecha/hora y transacción	**PASA**
10	UPDATE: solo 2 de 4 sentencias generan bitácora (cambios reales en columnas auditadas)	**PASA**
11	UPDATE captura valor anterior (1500.00) y nuevo (2500.75) de saldo	**PASA**
12	UPDATE registra usuario (bodeguero) y estación (PC-BODEGA-02)	**PASA**
13	UPDATE de 2 columnas genera 1 encabezado con 2 detalles	**PASA**
14	UPDATE captura estado ACTIVA -> BLOQUEADA	**PASA**
15	UPDATE de columna no auditada (notas) no se registra	**PASA**
16	UPDATE guarda la fila completa anterior (datos_anteriores)	**PASA**
17	DELETE: el registro ya no existe en la tabla origen	**PASA**
18	DELETE genera encabezado en bitácora	**PASA**
19	DELETE preserva la fila COMPLETA eliminada (incluye columnas no auditadas)	**PASA**
20	DELETE no tiene datos nuevos	**PASA**
21	DELETE guarda valor anterior de las 3 columnas auditadas	**PASA**
22	Historial del registro eliminado sigue disponible (INSERT + DELETE)	**PASA**
23	Vista v_registros_eliminados muestra el registro	**PASA**
24	Usuario sin privilegios no puede configurar	**PASA**
25	Escritura directa en bitácora denegada (usuario común)	**PASA**
26	Administrador tampoco puede borrar la bitácora	**PASA**
27	Bitácora inmutable incluso para superusuario	**PASA**
28	Valida existencia de tabla	**PASA**
29	Valida existencia de columna	**PASA**
30	RENAME COLUMN: la configuración se sincroniza automáticamente	**PASA**
31	DROP COLUMN de columna auditada es bloqueado por dependencia	**PASA**
32	DROP COLUMN: se marca la columna y el trigger se regenera sin romper DML	**PASA**
33	Tabla de configuración refleja el estado real del trigger	**PASA**
34	Consulta por usuario retorna su historial	**PASA**
35	Consulta por columna retorna solo cambios de esa columna	**PASA**
36	Consulta filtrada por operación funciona	**PASA**
37	Desactivar elimina el trigger	**PASA**
38	Reactivar vuelve a generar el trigger	**PASA**
39	Rendimiento: todas las operaciones masivas quedaron auditadas (183.000)	**PASA**

Rendimiento con volumen creciente

Tiempo (ms) de operaciones masivas sobre dos tablas idénticas, sin y con auditoría de 3 columnas.

Registros	Operación	Sin auditoría	Con auditoría	Sobrecosto por fila (ms)
1,000	DELETE	0.4	107.1	0.1067
1,000	INSERT	4.7	125.5	0.1208
1,000	UPDATE	2.9	112.0	0.1091
10,000	DELETE	4.0	1330.0	0.1326
10,000	INSERT	22.3	1197.2	0.1175

10,000	UPDATE	26.8	1257.9	0.1231
50,000	DELETE	16.0	6191.9	0.1235
50,000	INSERT	138.1	6094.2	0.1191
50,000	UPDATE	150.2	6280.6	0.1226

El sobrecosto por fila se mantiene constante (≈0,1–0,2 ms) al crecer el volumen: el costo es lineal. En una operación normal de una sola fila el usuario no percibe la diferencia; en cargas masivas el tiempo aumenta porque cada fila genera sus registros de bitácora.

Consulta sobre bitácora poblada	ms
Historial de un registro	98.5
Historial de un usuario	25.9
Historial de una columna (pocos cambios)	7.7
Últimos 500 cambios (pantalla Bitácora)	14.1